PART I · THE ARGUMENT

The Category Error

OPENING CASE

Two subscriptions, one correction

DATED: JUNE 2025 TO JUNE 2026. VENDOR ACTIONS, DATES, AND FIGURES ARE SOURCED. THE BUYING ORGANIZATION IS A COMPOSITE.

In the spring of 2025, a software team at a mid-size company was doing what thousands of teams were doing: paying twenty dollars per developer per month for an AI coding assistant called Cursor, and treating that line on the budget the way it treated every other software line. The tool was useful. The price was small. The arrangement was, on its face, a subscription like any other, filed next to the ticketing system and the design software and the password manager. Nobody in finance thought about it again after signature, because nothing about a twenty-dollar seat asks to be thought about.

Then, on June 16, 2025, the arrangement changed underneath them. Cursor revised its Pro plan so that the previous monthly allotment of five hundred requests against external models, with Sonnet models counting as two, became a twenty-dollar monthly pool of frontier-model usage billed at the underlying API rates. Use of Cursor's own Tab model and of models in Auto became unlimited. For light users the change was invisible. For the team's heaviest users, the people getting the most value, the monthly allowance was consumed in a matter of prompts, after which additional usage was priced at the same rates. Charges arrived that no one had planned for. On July 4, after three weeks of complaint, Anysphere's chief executive, Michael Truell, published an apology, promised refunds for the intervening period, and explained the change. The explanation was not an excuse. It was an account of arithmetic: the newest models consumed far more tokens per request on

long-horizon tasks than the flat price had been built to absorb, and Cursor had been absorbing the difference.[1]

The same pattern, in the same window, played out at a far larger scale. GitHub Copilot spent the year from June 2025 to June 2026 dismantling its own flat model in two acts. First, on June 18, 2025, it began enforcing monthly premium-request allowances and letting customers pay for usage beyond them. Then, on June 1, 2026, it moved standard billing from premium requests to credits denominated in tokens, priced at each model's published rate. The transition carried exceptions: annual Pro and Pro+ subscribers remained on premium-request pricing until their terms expired, completions and Next Edit Suggestions stayed included without consuming credits, and every plan retained a monthly credit allowance. Base subscription prices did not change. On its January 28, 2026 earnings call, four months before the change, Microsoft reported the paid Copilot base at over 4.7 million subscribers, up 75 percent year over year.[2]

The two corrections did not look alike from the buyer's seat. One arrived retroactively, under public pressure, with an apology and refunds. The other arrived on a published schedule, announced in advance, supported by tooling, across every plan.[3] Set the difference aside and what remains is the same event twice. A product had been sold, and bought, as software: a seat, a flat price, a cost fixed at signature. The

---

1. Michael Truell, "Clarifying Our Pricing," *Cursor*, July 4, 2025, accessed July 29, 2026; Maxwell Zeff, "Cursor Apologizes for Unclear Pricing Changes That Upset Users," *TechCrunch*, July 7, 2025, accessed July 29, 2026. The Cursor post describes the change of June 16 and the clarification of June 30. The TechCrunch report carries Truell's role and title.

2. GitHub, "Update to GitHub Copilot Consumptive Billing Experience," *GitHub Changelog*, June 18, 2025, accessed July 28, 2026; GitHub, "GitHub Copilot Is Moving to Usage-Based Billing," *The GitHub Blog*, April 27, 2026, accessed July 29, 2026; Microsoft Corporation, "FY26 Second Quarter Earnings Call," January 28, 2026, accessed July 29, 2026. The Microsoft earnings call carries the subscriber figure and the growth rate.

3. GitHub, "Update to GitHub Copilot Consumptive Billing Experience," *GitHub Changelog*, June 18, 2025, accessed July 28, 2026; GitHub, "GitHub Copilot Is Moving to Usage-Based Billing," *The GitHub Blog*, April 27, 2026, accessed July 29, 2026. Act one sits on the GitHub Changelog, dated June 18, 2025. Act two was announced April 27, 2026, with a preview bill experience launched in early May ahead of the June 1 transition.

thing underneath the seat turned out to obey a different law. It consumed a resource on every use, the resource had a real and variable cost, and heavy use ran that cost past the flat price. When the gap grew wide enough, the arrangement changed. It changed on the provider's schedule, not the buyer's. It changed whether or not the buyer had ever looked at a meter. And it changed in one direction only, toward an entitlement indexed to consumption.

The buyers in these stories did not make an error of vendor selection. Cursor and Copilot were, and are, excellent tools. The buyers made an error of category. They filed a metered resource under the mental model they use for licensed software, and the mental model held right up until the moment it did not.

1.1 The purchase that is not one

The buyer brought a model to the transaction, and it was a good model, refined over three decades of enterprise software procurement. In that model, software is a license. The organization buys the right for a defined number of people to use a program. The marginal cost of an additional use, once the seat is paid for, is approximately zero: a licensed user who runs the program a thousand times a day costs the buyer the same as one who runs it once a week. Cost is therefore fixed at signature. The negotiation that matters happens once, at purchase, over the number of seats and the price per seat. That per-seat figure is the **access price**. After that, management of the software is

DEFINITION

Access price

The stated price of access to an AI capability, per seat or per subscription, distinct from the total cost of consuming the resource behind it.

DEFINITION

Software access model

The economic model, inherited from licensed software, in which cost is fixed at signature and the marginal cost of additional use is approximately zero; management is management of access.

management of access: who has a login, how many seats are active, when the contract renews.

This model, the **software access model**, is not wrong about licensed software. It is wrong about the thing now being sold under the same packaging. What the organization operates when it deploys an AI assistant is not a program that people are licensed to run. It is work that consumes a metered resource on every task. Each use sends a request to a model, the model performs computation to answer it, and that computation has a cost that scales with the size of the request and the size of the answer. The marginal cost of an additional use is not approximately zero. It is the central fact.

The consequence is a mismatch between what the organization manages and what the organization operates. Access management asks how many people may use the tool. Resource management asks how much of the resource the work consumes, at what cost, for what return. An organization that manages access while operating a metered resource is managing the wrong quantity. It counts seats while the bill is written in tokens, requests, and credits. The mental model determines what gets managed, and the wrong model manages the wrong thing.

1.2 The consumption event

The discipline that follows in this book is built on a single atomic unit. That unit is the **consumption event**. Every deployed use of AI, at bottom, is a stream of these events. They are the thing the meter counts, the thing the invoice sums, and the thing every later chapter learns to source, record, attribute, budget, allocate, and hold accountable.

DEFINITION

Consumption event

A request that consumes computational resource, metered in tokens or their equivalents, at a per-event cost greater than zero. The atomic unit of the discipline.

The anatomy of one event is simple, and Figure 1.1 sets it out. Something goes in: a prompt, a document, a conversation history, retrieved reference material. The model performs computation. Something comes back: a completion, a suggestion, an answer, a tool call. A meter records what was consumed, typically the count of input tokens and output tokens, the units into which a model divides the text it reads and writes, and sometimes the calls to retrieval or tools that the event triggered. And a record of that consumption is held somewhere. In the ordinary case it is held by the provider, and the buyer never sees it at all. What reaches the buyer is a summation.

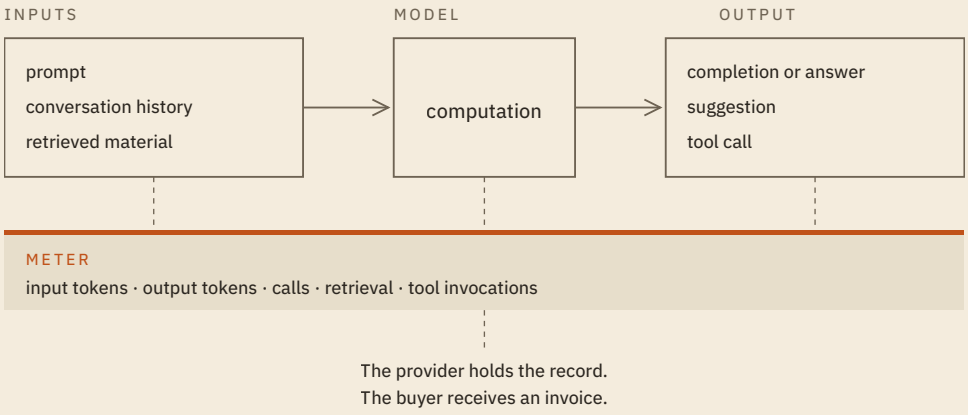

Figure 1.1 *Anatomy of a consumption event. Input is assembled, the model computes, output returns, and a meter records the resource drivers beneath. In the ordinary case the record stays on the provider's side, and what reaches the buyer is a summation rather than a record.*

The choice of unit is not arbitrary, and a different choice would produce a different discipline. Three candidates present themselves. The user is the unit the seat model already counts, and it fails immediately, because two users on identical seats can consume resource that differs by an order of magnitude. The task is the unit the business cares about, and it is the right unit for measuring value, but a single task may trigger one model call or four hundred, so it does not track cost. The event is what remains. It is the smallest thing that has a cost, and it is the only candidate the provider's meter actually records. A unit the meter does

not record cannot be reconciled against a bill, and a discipline whose unit cannot be reconciled against the bill is a discipline of assertions. Later chapters aggregate events upward into flows, workloads, and workflows, and they can do so precisely because the aggregation starts from something counted rather than estimated.

Figure 1.2 *Two purchase models. Under the seat model, cost is flat against rising usage. Under the event model, cost is the accumulating sum of consumption events. The same usage produces a flat cost on the left and a rising cost on the right.*

The seat model and the event model are not two views of the same economics. They are two different economics, and Figure 1.2 draws the difference the invoice hides. A horizontal line does not become an accumulating area because the contract calls it one. A buyer operating on the left panel while the right panel governs the bill will be surprised, and the surprise arrives as a number, on a schedule the buyer did not set.

1.3 The flat-rate objection, answered

One objection stands against everything said so far: "We pay thirty dollars per seat per month, the price is flat, we are never billed by the token, and for us this simply is software."

The objection is not confused, and the answer is not that the buyer is billed by the token after all. The answer is **meter relocation**: a flat rate moves the meter, it does not abolish it. When a provider offers a flat price for a metered resource, the provider does not stop metering. The

provider meters exactly what the buyer has declined to see, and prices the flat rate against an assumption about how much the average buyer will consume. That assumption is a bet. It holds while usage stays near the average the price assumed. It fails when usage is skewed, when a minority of heavy users consume a majority of the resource, because the flat price collected from everyone no longer covers the resource consumed by the few. At that point the arrangement is unstable, and the instability resolves in one direction. The provider re-indexes what the buyer receives to what the buyer consumes. It may do so by changing the price, by metering usage above an allowance, by changing the unit the allowance is denominated in, or by capping consumption outright. Chapter 4 sets out the full menu of instruments. What does not vary is the direction, or who chooses the moment: the correction arrives on the provider's schedule, because the provider holds the meter and the buyer does not.

This is not a claim about badly run vendors. It is a structural result, and the registry states it as a theorem.

THEOREM 1 · THM-009

AI Use Is Resource Consumption, Not Merely Software Access

Within a defined deployment and cost boundary, if:

(i) an AI activity requires compute, and executing it consumes resources;

(ii) production scaling expands the surface over which that activity is used;

(iii) the buyer's total cost extends beyond the access price; and

(iv) measurement makes resource use, or the cost-bearing activity, visible;

then that AI use is a resource-consuming operating activity, not merely software access.

The word carries a precise sense here. A theorem in this book is a statement proved within a stated system, from lemmas and propositions, not an empirical generalization drawn from the episodes above, and it binds only where its antecedents hold. The theorem does not say that a flat rate is impossible or that every subscription will be repriced tomorrow. It says something narrower and more durable: that in a deployed business context, where the activity requires compute, where scaling expands the surface of use, and where total cost runs past the access price, the thing being operated is a resource-consuming activity rather than software access. A flat rate packages that activity for sale. It is not evidence that the activity is something else, and the buyer who reads it as proof of software access has mistaken the packaging for the contents.

The public record already contains the pattern running its course, in more than one form.

> DATED: JANUARY 2025
>
> OpenAI's chief executive, Sam Altman, stated publicly that the company was losing money on its two-hundred-dollar Pro subscriptions because subscribers used them more than the price had assumed, and added that he had set the price himself.[4]

4. Sam Altman (@sama), post on X on ChatGPT Pro subscription economics, January 5, 2025, accessed July 28, 2026; Kyle Wiggers, "OpenAI Is Losing Money on Its Pricey ChatGPT Pro Plan, CEO Sam Altman Says," *TechCrunch*, January 5, 2025, accessed July 29, 2026. The TechCrunch report reproduces both statements.

DATED: JULY 2025

Claude Code subscribers found the usage limits already governing their plans abruptly tightened, with no announcement, many of them unaware that limits applied at all, and Anthropic acknowledged the reports without confirming a change. Eleven days later the company announced two weekly caps stacked on top of the existing five-hour limits, with the new constraints taking effect the following month rather than at any subscriber's renewal. Subscribers on the highest tier could buy additional usage at standard API rates.[5]

The two episodes that opened this chapter belong to the same record. Cursor and GitHub Copilot reached the same destination by different means, across two widely used AI coding subscriptions, inside twelve months. The flat rate did not make the meter disappear. It moved the meter to the provider's side of the table and moved the correction to the provider's calendar.

DEFINITION

Meter relocation

A flat rate does not abolish the meter but moves it to the provider's side, where the provider prices the flat rate against assumed consumption and corrects it on its own schedule when usage is skewed.

1.4 What follows if this is true

Accept the category, and a great deal follows without further argument. Resources that flow through an organization have long been the object of a mature management discipline, and the reader already knows its shape, because every organization that handles physical goods practices it daily. A manufacturer does not manage its steel by counting the people licensed to touch it. It sources the steel against stated require-

5. Anthropic (@AnthropicAI), post on X announcing new weekly rate limits for Claude Pro and Max, July 28, 2025, accessed July 28, 2026; Maxwell Zeff, "Anthropic Unveils New Rate Limits to Curb Claude Code Power Users," *TechCrunch*, July 28, 2025, accessed July 29, 2026; Russell Brandom, "Anthropic Tightens Usage Limits for Claude Code Without Telling Users," *TechCrunch*, July 17, 2025, accessed July 29, 2026. The announcement set two weekly caps effective August 28, 2025, added to five-hour limits already in force and tightened on July 17, 2025.

ments rather than accepting whatever the incumbent supplier offers. It plans and budgets the tonnage it expects to consume, and it notices when consumption departs from the plan. It records where the material goes as it moves through the plant, so that the quantity consumed by one product line can be distinguished from the quantity consumed by another. It allocates the material under a stated rule when supply is short, rather than letting the loudest line take what it wants. And it holds the finished output accountable for the material it consumed, so that a product whose margin cannot survive its own input cost is identified as such.

The same five practices, stated in timeless form, are not techniques belonging to manufacturing. They are the questions that any organization must be able to answer about anything that flows through it and costs money as it flows. What is being bought, and against what requirement. How much the organization expects to consume, and how it will know when the expectation fails. Where the resource actually went. Who decides when there is not enough. What the consumption returned. An organization that cannot answer those questions about a material flow is not managing that flow, whatever its org chart says. It is receiving invoices and paying them.

Deployed AI is such a resource. It flows through the organization as a stream of consumption events. It costs money as it flows. Its volume is driven by the volume of the work, which is to say by the very success of the deployment. Yet no assembled discipline governs it, and the reason is worth stating precisely, because it is not indifference. Each of the five questions falls inside the scope of some practice the organization already has. Procurement buys the contract. Cloud cost management owns the infrastructure bill. Engineering instruments what it operates. Finance allocates what it can trace. Every one of those practices was built around a boundary drawn before this resource existed, and a resource that crosses all five boundaries is owned, in practice, by none of them. Chapter 14 draws the border between this book's subject and the

nearest of those neighbors. What is absent is not attention. It is assembly.

Organizations that would never let a material flow through the plant unrecorded let the AI flow through the work unrecorded.

The failure to answer those questions is not abstract. It shows up in three places, and the reader can look for all three in any organization, including the reader's own. The first is the quantity managed. An organization operating on the seat model tracks headcount, license utilization, and renewal dates, and none of those three quantities moves with the bill. The second is the record. Under the seat model there is little worth recording, because a licensed use costs nothing at the margin. Under the event model every use has a resource footprint, but the buyer who never built a meter holds no record of it. An invoice is not that record; an invoice is its summation. It reports a total without reporting what produced it. The third is the anchor for accountability. A cost that cannot be traced to the work that incurred it cannot be assigned to anyone, and a cost assigned to no one is defended by no one.

It would be a mistake to read those absences as negligence. They are inheritance, and it is the same inheritance that left the resource unowned. The organization did not decide to stop metering. It never began, because the thing it had been buying for three decades never required it. Enterprise software procurement produced a management apparatus tuned precisely to the economics of the license, and that apparatus worked. It was then carried across to a purchase that arrived in the same packaging, through the same channel, at a price small enough to clear approval without argument. The apparatus did not fail. It was applied to a resource it was never built to govern, and it went on reporting faithfully about a quantity that had stopped governing anything.

What ends the arrangement is scale. A category error is affordable while the sums are small, and the sums stayed small for as long as deployment meant a pilot: one team, a few licenses, a bill that rounded to nothing against the software budget. The error becomes expensive at

the moment the pilot becomes production, because production multiplies consumption events by the volume of the work itself, and the work is not small. The contact-center deployment inventoried later in this chapter runs on roughly five thousand seats and generates tens of millions of consumption events a month, a ratio the craft section works out in full. At pilot scale, the seat count is a harmless simplification. At production scale, it is a blindfold.

The stakes of this chapter are therefore the stakes of the book. A resource that flows needs a discipline, and none exists in assembled form, so building it is the work of the remaining chapters. The next of them takes the atomic unit defined here and asks what becomes visible when many events are seen at once. That is the first step from a unit of cost to something an organization can manage.

1.5 What this book is not

Three subjects sit close enough to this one to be mistaken for it, and none of them is it. This book is not about the internals of models. The reader will find no account here of architectures, training, or weights. It is not about prompt engineering, and it offers no techniques for extracting better outputs from a given model. It is not about use-case ideation. It will not help the reader decide which problems to point AI at. Those are real subjects with their own literatures, and Chapter 3 draws the exact line between each of them and this one. What remains after the exclusions is the whole of the subject: the economics of the resource once the decision to deploy has been made, what it consumes, what it costs, and how an organization governs both.

CRAFT SECTION

The consumption-event inventory

WORKED EXAMPLE SETTING DRAWN FROM THE CITED STUDY. EVENT ARCHITECTURE AND VOLUME ASSUMPTIONS STIPULATED FOR THIS EXERCISE AND LABELLED WHERE USED.

The **consumption-event inventory** turns the definition given in 1.2 into a procedure. It takes a description of an AI deployment and produces a structured account of what the deployment consumes, where the meter sits, and what the seat count conceals. It is the first artifact the reader produces, and every later artifact in this book assumes it.

The procedure has four steps. **Step 1, enumerate the event types.** Given a deployment description, list the distinct kinds of consumption event the deployment generates. A deployment rarely produces one kind of event; it produces several, each with its own trigger and its own resource profile. **Step 2, identify each type's resource drivers.** For each event type, name what actually consumes resource: input tokens, output tokens, the number of calls, retrieval operations, and tool invocations. The driver is what the bill scales with, and it is almost never the number of people. **Step 3, locate the meter.** State where the metering happens and who holds the record. In the ordinary case the meter is on the provider's side, and what the buyer receives instead of a meter is an invoice. **Step 4, state what the inventory reveals that the seat count conceals.** Name the quantity that actually drives cost, and set it against the quantity the organization currently tracks.

Consider a customer-support organization that has deployed a generative AI assistant across its contact center. For each incoming customer message, the assistant drafts a suggested reply that the agent may edit and send; the assistant draws on a knowledge base and on the conversation so far. The deployment covers a large agent population, on the

order of five thousand agents, and the organization currently accounts for it as a per-seat tool.[6]

MODEL INVENTORY

Step 1. Event types. The deployment generates at least three: (a) a suggested-reply generation, triggered each time an agent asks the assistant to draft a response to a customer message; (b) a knowledge retrieval, triggered when the assistant pulls reference material to ground a suggestion; and (c) a conversation-close operation, where the deployment summarizes or tags the resolved conversation.

Step 2. Resource drivers. For suggested-reply generation, the drivers are input tokens (the system instructions, the retrieved knowledge, and the conversation history assembled into the prompt) and output tokens (the drafted reply). The volume driver is the number of customer messages that receive a drafted reply, which scales with contacts multiplied by the number of turns per contact. For knowledge retrieval, the drivers are the number of retrieval calls and the tokens those calls embed and return. For the conversation-close operation, the drivers are input tokens (the full conversation) and output tokens (the summary or tags), scaling with the number of resolved conversations.

Step 3. The meter. The generation and close operations are metered on the provider's side, by token. Retrieval is metered wherever the retrieval service runs. In the architecture stipulated here that is the provider's managed index, metered by call and by token; an organization that operates its own vector store meters that step itself. Meter location is a property of the architecture rather than of the event type, which is why Step 3 directs the reader to locate the meter instead of assuming it. What the organization receives is a monthly invoice, and beneath its per-seat plan, a seat count.

Step 4. What the seat count conceals. The seat count treats the agent population as a set of equal units: five thousand seats at one price. Put the volume drivers from Step 2 against it and the two quantities separate immediately. Stipulate, for this exercise, a round five thousand agents, each handling 40 contacts on a working day, with 6 turns per contact receiving a drafted reply, one retrieval per drafted reply, and 21 working days in the month. Suggested-reply generations then run at 5,000 × 40 × 6 × 21, or 25.2 million events a month. Retrievals match them one for one at 25.2 million. Conversation-close operations run once per contact, at 4.2 million.

6. Erik Brynjolfsson, Danielle Li, and Lindsey Raymond, "Generative AI at Work," *Quarterly Journal of Economics* (2025). The study supplies the setting and the scale of the agent population. The event architecture and the volumes used below are stipulated for this exercise and are not reported by the study. It returns in Chapter 6 as the book's anchor case on realized value.

The deployment bills against roughly 54.6 million consumption events a month, which is about 10,900 events behind every seat.

Change any stipulated figure and the total moves; change none of them and the seat count still does not move, because the seat count is not a function of any of these quantities. Two agents on identical seats can consume resource that differs by an order of magnitude: a high-volume agent handling long, reference-heavy conversations generates many times the consumption of a low-volume agent, at the same seat cost. The quantity that drives the bill is tokens per resolved contact. It does not appear on the seat line, and until the inventory names it, the organization is managing a number that does not govern its cost.

The inventory has done its work when the organization can see, for the first time, the shape of what it is buying. Not five thousand seats. A flow of tens of millions of consumption events, whose volume and intensity, not whose headcount, determine the bill.

Chapter summary

The category error has a name and a shape. Under the software access model, cost is fixed at signature and management is management of seats. Under the resource consumption model, cost is the accumulating sum of metered events and management is management of a flow. The reader can now tell the two apart, and can define the consumption event as the discipline's atomic unit: the smallest thing that has a cost, and the only candidate the meter records. The flat-rate objection has an answer that does not depend on predicting any vendor's behavior. A flat price relocates the meter to the provider; it does not abolish it. Behind that answer stands a structural result, THM-009, that a deployed AI activity is resource-consuming operating activity rather than software access. And the reader can run the consumption-event inventory against a deployment description, producing the event types, their resource drivers, the location of the meter, and the cost driver the seat count conceals. One thing the reader cannot yet do is manage the flow the inventory reveals. Seeing it whole comes first.

Key terms

Consumption event

A request that consumes computational resource, metered in tokens or their equivalents, at a per-event cost greater than zero. The atomic unit of the discipline.

Resource consumption model

The economic model in which deployed AI is a metered resource consumed on every task, with cost accruing per event; contrasted with the software access model.

Software access model

The economic model, inherited from licensed software, in which cost is fixed at signature and the marginal cost of additional use is approximately zero; management is management of access.

Access price

The stated price of access to an AI capability, per seat or per subscription, distinct from the total cost of consuming the resource behind it.

Metered resource

A resource whose consumption is measured per unit of use, here in tokens or their equivalents.

Flat-rate objection

The claim that a flat per-seat price makes AI equivalent to licensed software; answered by meter relocation.

Meter relocation

The principle that a flat rate does not abolish the meter but moves it to the provider's side, where the provider prices the flat rate against assumed consumption and corrects it on its own schedule when usage is skewed.

Discussion questions and problems

DISCUSSION QUESTIONS

1. A colleague argues that because your organization pays a flat per-seat price and never sees a token bill, AI is a seat-priced good for you and the consumption framing does not apply. Explain, without appealing to any specific vendor's future behavior, why the per-seat contract does not make AI a seat-priced good.

2. The Cursor and GitHub Copilot episodes are often read as stories about two particular companies changing their prices. Explain what the two episodes, taken together, reveal about where the meter was all along, and why the pairing is harder to dismiss than either episode alone.

3. Section 1.2 argues that the event, rather than the user or the task, is the right atomic unit for this discipline. Construct the best case you can for the task as the unit instead. What would a discipline built on tasks be able to say that an event-based one cannot, and what would it lose?

4. Construct the strongest version of the flat-rate objection yourself, in the voice of a capable skeptic who has read this chapter. Then answer it. Your answer should not depend on predicting that any particular subscription will be repriced.

PROBLEMS

P1 · WORKED

The CIO memo

A board member has written to the chief information officer arguing that the company is overcomplicating a simple software purchase and should stop

tracking AI usage as if it were a utility. Write the CIO's one-page reply defending consumption economics against the flat-rate objection.

MODEL REPLY

Thank you for the note. I want to agree with the part of it that is right and then explain the part that will cost us if we accept it. You are right that our current AI tools are billed to us at a flat per-seat price, and that this looks exactly like the software we have always bought. The reason I am tracking usage anyway is that the flat price is not a statement about our costs; it is a bet the vendor has made about our consumption. Behind the flat price, every use of these tools consumes a metered resource that costs the vendor real money, and the vendor prices the flat rate against an assumption about how much we will use. While our usage stays near that assumption, nothing happens. When our heaviest teams pull usage above it, the vendor's arrangement stops covering its costs, and the vendor corrects it. Sometimes that means a higher price, sometimes a cap on how much we may consume, sometimes a change to what the plan includes. We have watched this happen across two widely used AI coding subscriptions inside a single year. Tracking our own usage is not utility theater. It is how we see the correction coming before it arrives as a number we did not plan for, and how we keep the timing of that correction from being entirely the vendor's to choose. I am not asking us to manage AI like plumbing. I am asking us to stop managing a metered resource as if it were a license, because the two behave differently exactly when the stakes are highest.

ANNOTATED REASONING

The reply concedes the true premise, that the price is flat and looks like software, before contesting the inference, which disarms the objection rather than dodging it. It relocates the meter, treating the flat price as a bet against assumed consumption, rather than claiming a token bill the reader knows does not exist. It grounds the structural claim in the documented pattern rather than in a prediction about a specific vendor, satisfying the evidence standard. And it closes on the buyer's actual interest, the timing of the correction, which is the concrete thing consumption tracking buys.

P2 · WORKED

Inventory a coding-assistant deployment

A software organization has deployed an AI coding assistant to three hundred developers. The assistant offers inline code completions as developers type, answers chat questions in the editor, and runs multi-step agentic tasks on request. Produce the consumption-event inventory.

MODEL INVENTORY

Event types: (a) inline completion, triggered continuously as developers type; (b) editor chat query, triggered when a developer asks a question; (c) agentic task run, triggered when a developer delegates a multi-step task.

Resource drivers: inline completion is driven by input tokens (the surrounding code context sent for each completion) and output tokens (the suggested code), with a very high call count because completions fire constantly; chat query is driven by input tokens (the question plus code context) and output tokens (the answer); agentic task run is driven by input and output tokens across many model calls per task, plus tool invocations, and is the highest-intensity event by a wide margin.

Meter: provider side, per token, with agentic runs additionally consuming tool calls; the buyer receives an invoice, and under a seat plan, a seat count.

What the seat count conceals: cost is driven by completion frequency and, above all, by agentic run volume and length, not by the number of developer seats. A single developer who leans on agentic runs can consume more resource than a dozen who use only inline completion, at the same seat cost. The cost driver is tokens per developer per day, concentrated in agentic runs, and the seat line shows none of it.

P3 · COMPLETION

Inventory a document-review deployment

A legal operations team has deployed an AI assistant that, for each contract submitted, extracts key clauses, compares them against a policy playbook, and drafts a redline memo. Complete the inventory below by filling the blank column.

EVENT TYPE	RESOURCE DRIVERS	METER LOCATION	WHAT THE SEAT COUNT CONCEALS
_____	Input tokens (full contract text), output tokens (extracted clauses); scales with contract length	Provider side, per token	Cost scales with document length, not reviewer headcount
_____	Input tokens (extracted clauses plus playbook), output tokens (comparison result); retrieval calls against the playbook	Provider side, per token and per call	A long playbook inflates every comparison regardless of seat count
_____	Input tokens (clauses, comparison, contract context), output tokens (the drafted memo); highest output-token event	Provider side, per token	The drafting step, not the reviewer, is the dominant cost driver

Interleaving: none. This is the first chapter; problem sets begin reaching back to earlier chapters in Chapter 2.